

VHDL Programming: UNIT 2

By: Dr. Yogita Arora
Associate Prof., ECE dept.

Subprograms and Overloading.

SO-1

SUBPROGRAMS

→ defines a sequential algorithm that performs certain computation.

2 Kinds of subprogram

Functions

→ used for computing single values.

→ Executes in zero simulation time

Procedures

→ returns zero or more values.

→ may or may not execute in zero simulation time.

Format (syntax):

subprogram-specification is

subprogram-item-declarations

begin

subprogram-statements

end [function/procedure] [subprogram-name].

FUNCTIONS

- returns single value.

return [expression];

- runs in zero simulation time.
- Common uses are as
 - Resolution Function
 - Type conversion function.

eg:

Function TO_CHARACTER (ARG: STD_ULOGIC)

return CHARACTER is

begin

Case ARG is

when 'U' => return 'U';

when 'X' => return 'X';

when '0' => return '0';

when '1' => return '1';

when 'Z' => return 'Z';

when 'W' => return 'W';

end Case;

end TO_CHARACTER;

Pure Function:

SO-2.2

Function Largest (Total : integer; Set : Pattern)

-- Pattern is 1D array defined elsewhere (N.N. no.s)

Variable Return_value : Real := 0.0;

begin

For K in Set's range loop

if Set(K) > Return_value then

Return_value := Set(K);

end if;

end loop;

return Return_value;

end Largest;

Impure function:

impure function Random (seed : Real) return real is

variable Num : real;

attribute foreign of Random : function is "NUM =
rand(seed)";

begin

return Num;

end Random;

Function that performs Byte Reversal

SO-3
to 3.2

Function Byte_Reversal (Arg: std_logic_vector)
return std_logic_vector is

Variable RESULT:

std_logic_vector (Arg' Reverse_Range);

Attribute of
signal

Begin

For J in Arg' Range loop

Attribute
of signal

RESULT (J) := Arg (J);

end loop;

return RESULT;

end Byte_Reversal

Function call:

Byte_Reversal ("11001110")

PURE & IMPURE FUNCTIONS:

A PURE function is one that returns the same value each time the function is called with same set of actuals.

An IMPURE function is one that potentially returns different value each time it is called with the same set of actuals.

eg: NOW.

PROCEDURE:

- Allow decomposition of large behaviors into modular sections.
- can return zero or more values using parameters of mode out & inout.
- Syntax (specification of Procedure Body).
Procedure procedure-name (parameter list)

For procedure call

[Label:] procedure-name [list-of-actuals];

→ A procedure body can have wait statement SO-4
while function cannot.

eg:

type op_code is (ADD, SUB, MUL, DIV, LT, LE, EQ);

Procedure ARITH_UNIT (A, B : in INTEGER; OP : in
op_code; Z : out INTEGER; ZCOMP : out BOOLEAN) is
begin

Case OP is
 when ADD $\Rightarrow$ Z := A + B;
 when SUB $\Rightarrow$ Z := A - B;
 when MUL $\Rightarrow$ Z := A * B;
 when DIV $\Rightarrow$ Z := A / B;
 when LT $\Rightarrow$ ZCOMP := A < B;
 when LE $\Rightarrow$ ZCOMP := A <= B;
 when EQ $\Rightarrow$ ZCOMP := A = B;

end case;
end ARITH_UNIT;

Another example:

→ Procedure rotates the specified signal vector Array-name, starting from bit start-bit to bit stop-bit, by the ROTATE BY value.

Procedure ROTATE_LEFT

(signal ARRAY_NAME : inout BIT_VECTOR;
START_BIT, STOP_BIT : in NATURAL;
ROTATE_BY : in POSITIVE) is

Variable FILL_VALUE : BIT;

begin

Assert STOP_BIT > START_BIT
report "STOP_BIT is not greater than start-bit."
severity NOTE;

For MACVAR3 in 1 to ROTATE_BY loop

FILL_VALUE := ARRAY_NAME (STOP_BIT);

For MACVAR1 in STOP_BIT downto START_BIT + 1

ARRAY_NAME (MACVAR1) <= ARRAY_NAME
(MACVAR1 - 1);

end loop;

ARRAY_NAME (START_BIT) <= FILL_VALUE;

end loop;

end procedure ROTATE_LEFT;

→ A sequential procedure call statement is executed sequentially statements surrounding it.

A concurrent procedure call statement is executed whenever an event occurs on one of the parameters i.e. a signal of mode in or inout.

Declarations:

- A subprogram body may appear in the declarative part of the block in which a call is made.
- Package Body contains subprogram body and package declaration contains the corresponding subprogram declaration.

Syntax: (for subprogram declaration)

subprogram-specification;

procedure ARITH_UNIT (A, B : in INTEGER; OP : in OP_CODE;
Z : out INTEGER; ZCOMP : out BOD-
-LEAN);

Function LARGEST (TOTAL_NO : INTEGER; SET : PATTERN)
return REAL;

- Two subprogram can call each other recursively but, that subprogram should at least declared first before calling.

Procedure PROC-P (....)

variable LOCAL :

begin

....
LOCAL := FUNC-Q(B);

....
end PROC-P;

function FUNC-Q (....)

....
PROC-P (....);

....
end FUNC-Q;

-- illegal fⁿ call
-- (∵ func-q has not
-- yet declared).

Subprogram Overloading:

50-8

When two or more subprograms have same name, it is said to be overloaded.

eg:

1st: function COUNT (ORANGES: INTEGER) return INTEGER;

and: function COUNT (APPLES: BIT) return INTEGER;

→ Functions are overloaded. $\hat{=}$ function call,
COUNT(20) for 1st fⁿ. $\hat{=}$ COUNT('1') for 2nd fⁿ.

→ If two overloaded subprograms have same number of parameter types $\hat{=}$ result types, it is possible for one subprogram to hide the other.

Architecture HIDING of Dummy-entity is

f1 → function ADD (A, B: Bit_vector) return Bit_vector is
begin

-- Body
end ADD;

begin SUM_IN_ARCH <= ADD(IN1, IN2);

process function ADD (C, D: Bit_vector) return bit-vector is

f2 → begin
-- Body
end ADD;

begin SUM_IN_PROCESS <= ADD(IN1, IN2)

SUM_IN_ARCH <= HIDING.ADD (IN1, IN2);

end process;
end HIDING;

f2 hides f1.

→ But, function can still be accessed by qualifying the function name with architecture name.

→ Also, possible for two overloaded subprograms to be directly visible within a region, by using USE clauses.

eg:

package P1 is
function ADD (A, B: Bit_vector) return
Bit_vector;

end P1;

package P2 is
function ADD (X, Y: Bit_vector) return Bit_vector;
end P2;

USE WORK.P1.all, WORK.P2.all;
Architecture OVERLOADED of DUMMY-ENTITY is
begin Named Association

-- stmt1 sum_correct <= ADD (X => IN1, Y => IN2);

-- stmt2 sum_error <= ADD (IN1, IN2); -- An error

end OVERLOADED;

→ stmt2 is an error.

→ Two overloaded subprograms can have a different number of parameters.

Exact subprogram being called using the following information: 50-10

- I Subprogram name.
- II Number of actuals.
- III Types and order of actuals.
- IV Names of formal parameters. (Named Association)
- V Result type. (for functions)

→ Overloading does not distinguish by subtypes.

eg: type logic5 is ('x', '0', '1', 'D', 'Z');
subtype logic5_01 is logic5 range '0' to '1';

f1 function Rising-edge (signal clock: logic5)
return Boolean;

f2 function Rising-edge (signal clock: logic5_01)
return Boolean;

f2 hides f1

⊥

signal CK: logic5;

... Rising-edge (CK) ... -- An error.

→ Overloading does not distinguish by parameter modes.
eg: procedure P1 (A, B: in bit; C: out Bit);
 procedure P1 (A, B: in bit; C: inout Bit);

II hides I

operator Overloading:

- when a standard operator symbol is made to behave differently based on the type of its operands, the operator is said to be overloaded
- Possible to augment the and operation as a function that operates on arguments of type MVL

type MVL is ('0', '1', '0' and 'Z');

subtype S1, S2 is MVL;

S1 and S2; -- refers to 'and' function
-- user defined.

CK1, CK2: Bit;

CK1 and CK2

-- refers to predefined and
-- operator.

- Number of parameters in such a function must match in cardinality with that of predefined operator.

eg: function "and" (L, R: MVL) return MVL;
function "not" (R: MVL) return MVL;

Overloaded functions, the operators can now be called using two different types of notations:

1. standard operator notation.
2. standard function call notation.

eg: signal A, B, C: MVL;
 signal X, Y, Z: BIT;

A <= 'Z' or '1';

B <= "or" ('0', 'Z');

Z <= (X and Y) or A;

-- standard operator notation.

-- function call notation.

-- An error::, no overloaded

-- or operators defined with the first parameter type of BIT
 -- and second parameter type of MVL.

Above problem can be resolved,

function "or" (L: Bit; R: MVL) return Bit;

→ Arithmetic operators not work on vectors

function "+" (op1, op2: Bit_vector) return
 Bit_vector;

function "-" (op1, op2: std_logic_vector)
 return std_logic_vector;

Signatures:

Why required?

→ To uniquely identify an overloaded subprogram
 & an overloaded enumeration literal.

-- Two overloaded enumeration type declaration.

type Mybit is ('0', '1');

type four-value is ('U', '0', '1', 'Z');

-- Two overloaded function operator fⁿ declaration

function "+" (A, B: bit_vector) return Bit_vector;

function "+" (A, B: std_logic_vector) return
 std_logic_vector;

→ Signatures have been provided as a means SO-13
of distinguishing overloaded subprograms and
overloaded enumeration literals.

≡ signature shows the parameter types and result
type of an overloaded subprogram or overloaded
enumeration literal

syntax:

[first-parameter-type, second-parameter-type ...
↓
compulsory [return function-return-value]].

eg:

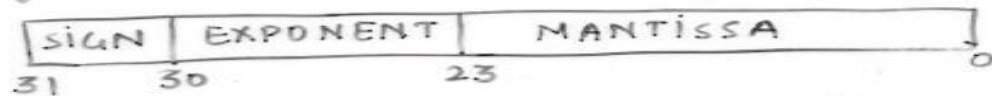
[bit-vector, bit-vector return bit-vector]

[std_logic_vector, std_logic_vector
return std_logic_vector]

→ signatures can be used in Alias declaration etc.
to uniquely identify an overloaded subprogram

Alias: shorthand notation for items that have
long names.

eg: signal s: bit_vector (31 downto 0);



Alias can be defined for each of required field.

Variable Data_word : Bit_vector (15 downto 0);

Alias Data_bus: Bit_vector (7 downto 0) is
Data_word (15 downto 8);

Alias status : Bit_vector (0 to 3) is Data_word (3 downto 0);

Alias Reset : Bit is Data_word (4);

Data_bus can be used whenever Data_word (15 downto 8) is needed and so on.

How Alias are used with signatures

Function TO_STDLOGICVECTOR (A: INTEGER; SIZE: INTEGER)

return STD_LOGIC_VECTOR;

Function "mod" (A, B: STD_LOGIC_VECTOR)

return STD_LOGIC_VECTOR;

Alias MOD_SLV is

"mod" [STD_LOGIC_VECTOR, STD_LOGIC_VECTOR
return STD_LOGIC_VECTOR];

Alias TO_S is TO_STDLOGICVECTOR [INTEGER, INTEGER

return STD_LOGIC_VECTOR];

Default Values for Parameters:

→ Possible to assign default values to input parameters in a subprogram declaration.

(If no actual is specified for formal parameter that has default value, then default value is used).

eg: function My-and (A: in Bit := '0', B: in Bit := '1') return Bit;

→ If default value is assigned, it is optional to pass an actual value in a subprogram call (Keyword "open" is used as an actual)

eg: My-and ();
My-and (B => sig-P) ...
My-and (B => sig-P, A => sig-Q) ...
My-and (A => sig-Q, B => open)

Another eg: Function To-time (Arg: integer; scale: Time := ns)
return Time is
begin
return Arg * scale;
end To-time;

Function calls:

To-time (56)
To-time (56, 10 ms)

Model simulation

- Environment necessary for simulating hardware models.

Simulation

Before beginning hardware modelling one has to decide about the values used for simulation.

Abstract Level - values are integers.

Nine value simulation is normally done.

'U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-' STD_ULOGIC

(Leftmost value is 'U' so if the value is not assigned 'U' is used).

→ For purely behavioural model

Library IEEE;

Use IEEE.STD_Logic_1164.all;

→ For structural model

- Use reference primitive components defined in certain Library.
- may use package
- behavioural models are required for primitive components.

And all the models should be compiled in same Library.

Test bench

MS-2

IS A MODEL THAT IS USED TO EXERCISE AND VERIFY THE CORRECTNESS OF A HARDWARE MODEL.

PURPOSE OF TEST BENCH

1. To generate stimulus for simulation (waveform).
2. To apply this stimulus to the entity under test & collect responses.
3. To compare output responses with expected values.

TEST BENCH FORMAT

entity TEST_BENCH is
end;
architecture TB_BEH of TEST_BENCH is
component entity under test
port (List-of-ports - their types - and modes);
end component;
local-signal-declarations;
begin
generate-waveform using behavioural constructs;
apply to E-U-T;
EUT: Entity-under-test port map (port-association);
monitor values compare results;
end TB_BEH;

Stimulus is automatically applied to E-U-T by instantiation and then specifying appropriate interface signals.

WAVEFORM GENERATION

MS-3

1. Create waveform and apply stimulus at certain discrete time intervals. (repetitive pattern is used).
2. Generate stimulus based on state of the entity i.e. based on O/P response of entity (sequential set of values).

REPETITIVE PATTERNS

- constant on-off-delay.

CLK <= not CLK after 5ns; CLK



- Varying ON-OFF period.

process
constant ON_PERIOD: TIME := 35ns;
constant OFF_PERIOD: TIME := 20ns;

begin
wait for OFF_PERIOD;
D-OUT <= '1';
wait for ON_PERIOD;
D-OUT <= '0';
end process;



To control the amount of time for simulation, one can use assertion statement.

assert NOW <= 1000ns

report "simulation completed successfully"

severity ERROR;

OR if NOW > 1000ns then

wait;

end if;

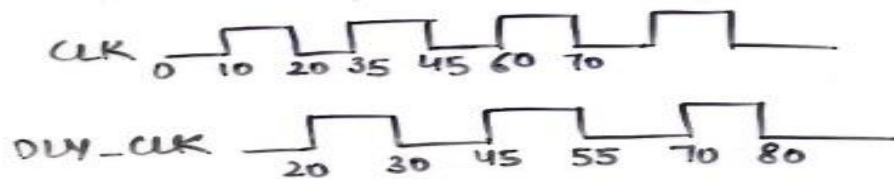
-- suspends process indefinitely.

Phase delayed clocks

MS-4

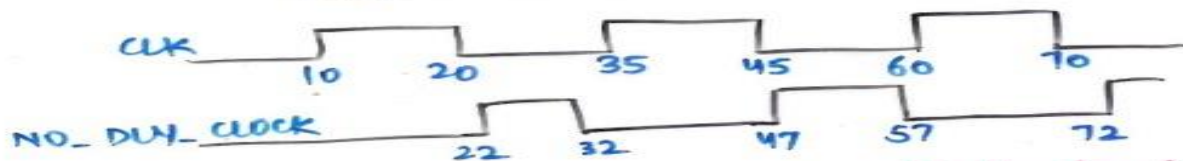
- can be generated using 'DELAYED' predefined attribute.

$DLV_CLK \leftarrow CLK \text{ DELAYED } (10\text{ns});$



nonoverlapping clocks

$NO_DLV_CLOCK \leftarrow CLK \text{ DELAYED } (12\text{ns});$



- Time specified in DELAYED attribute is larger than the clock's ON period.

SEQUENTIAL SET OF VALUES

- multiple waveforms in a concurrent signal assignment statement.

$RESET \leftarrow '0', '1' \text{ after } 10\text{ms}, '0' \text{ after } 30\text{ms}, '1' \text{ after } 55\text{ms};$

- If the set of values are required to be repeated the signal assignment statement can be placed inside the process stmt and wait for delay can be used.

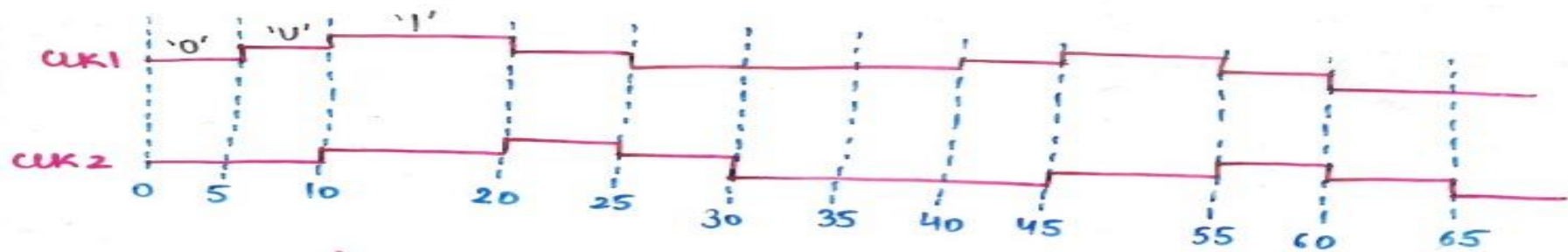
signal CLK1, CLK2: STD-LOGIC := '0';

...
P1: PROCESS

CLK1 <= '0' after 5ns, '1' after 10ns, '0' after 20ns, '0'
after 25ns;

CLK2 <= '0' after 10ns, '1' after 20ns, '0' after 25ns,
'0' after 30ns;

wait for 35ns;
end process;



Using Vectors

used for applying stimulus to a set of signals store
i/p vector in a table.

constant no.-of-bits: integer := 4;

constant no.-of-vectors: integer := 5;

type TABLE-type is array (1 to no.-of-vectors) of
std-logic-vector (1 to no.-of-bits);

constant input-vectors: TABLE-type :=
("1001", "1000", "0010", "0110");

constant vector-period: TIME := 100 ns;

signal inputs : std_logic_vector (1 to no.-of-bits);

signal A, B, C : std_logic;

signal D: std_logic_vector (0 to 1);

E.U.T has 4 i/p's if vectors are to be applied at regular time intervals then process is used.

Process

Variable J: POSITIVE;

begin

wait for vector_period * J;

INPUTS <= input_vectors (J);

J := J + 1;

if J > no.-of-vectors then

wait;

end if;

end process;

A <= INPUTS (1); B <= INPUTS (4); C <= INPUTS (1);

D <= INPUTS (2 to 3);

For arbitrary interval, concurrent signal assignment can be used.

INPUTS <= INPUTS_VECTORS (1) after 10 ns;

INPUTS <= INPUTS_VECTORS (2) after 25 ns;

INPUTS <= INPUTS_VECTORS (3) after 35 ns;

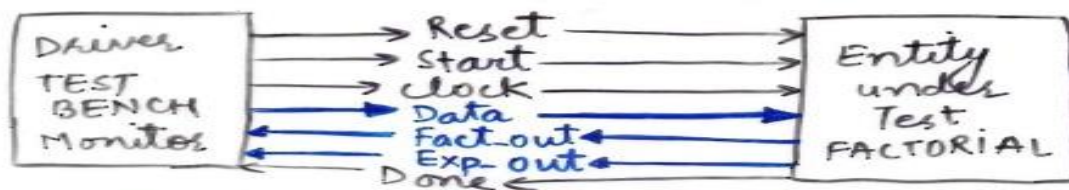
INPUTS <= INPUTS_VECTORS (4) after 40 ns;

inputs can also be specified in a file.

SECOND APPROACH

MS-7

Stimulus generation based on state of E.U.T.
used for FSM testing



Handshake between test bench and entity under Test.

- (i) Reset signal resets factorial model.
- (ii) Start signal is set after data is applied.
- (iii) Done signal is set to indicate completion of operation.

$$\text{RESULT} = \text{fac-out} \times 2^{\text{Exp-out}}$$

The test bench monitors the behaviours and apply different patterns based on output.

Library IEEE, BOOK_LIB;

Use IEEE.std_logic_1164.all, BOOK_LIB.UTILS_PKG.all;

- Functions to_integer and to_std_logic_vector
- are defined in Package UTILS_PKG.

entity FAC_TB is

constant IN_MAX: INTEGER := 5;

constant OUT_MAX: INTEGER := 8;

end FAC_TB;

Architecture FACTB_func of FACTB is
component FACTORIAL

MS-8

```
port( RESET, START, CLOCK : in std_logic;  
      Data; in std_logic_vector (IN_MAX-1 downto 0);  
      DONE : out std_logic  
      FOUT, EOUT : out std_logic_vector (OUT_MAX-1  
                                           downto 0));
```

end component;

type FAC_state is (RESET_ST, START_ST, APPL_DATA_ST,
 WAIT_RESULT_ST);

signal CLK, RESET, START, DONE : std_logic;

signal Data : std_logic_vector (IN_MAX-1 downto 0) :=
 (others => '0');

signal FAC_OUT, EXP_OUT : std_logic_vector (OUT_MAX-
 downto 0);

signal Next_state : FAC_state; -- starts with Reset-st
 -- state.

constant Max_Apply : POSITIVE := 20;

begin

CLK_P : Process

begin

CLK <= '0';

wait for 4ns;

CLK <= '1';

wait for 6ns;

end process CLK_P;

Process

Variable Num_applied : Positive;

-- starting
-- with 1.

MS-9

begin

if CLK='0' and CLK'EVENT then

case Next_state is

when RESET_st => RESET <= '1'; START <= '0';

NEXT_STATE <= APPL_DATA_ST;

when APPL_DATA_ST => DATA <= To_std_logic_vector
(Num_applied, IN_MAX);

NEXT_STATE <= START_ST;

when START_ST => START <= '1';

NEXT_STATE <= WAIT_RESULT_ST;

when WAIT_RESULT_ST =>

RESET <= '0'; START <= '0';

wait until DONE := '1';

Assert (NUM_APPLIED = TO_INTEGER (FAC_OUT) *
(2 ** TO_INTEGER (EXP_OUT)))

Report "Incorrect result from factorial model.";

NUM_APPLIED := NUM_APPLIED + 1;

if NUM_APPLIED < MAX_APPLY then

NEXT_STATE <= APPL_STATE_ST;

else

assert FALSE

report "Test completed successfully.";

end if;

end case;

end if;
end process;

-- Apply to entity under test.

FI: FACTORIAL port map (RESET, START, CLK, DATA,
DONE, FAC_OUT, EXP_OUT);

end FAC_TB_func;

Monitoring Behaviour

How Test Bench can monitor the behaviour of entity and apply different patterns based on output response?

→ To monitor the output response of an entity is to apply a vector, sample the output after a specific time and then verify to make sure that output response matches the expected value.

Library IEEE;

Library BOOK_lib;

Use IEEE. std_logic_1164.all;

Use BOOK_lib. Utils_pkg.all;

entity Test is end;

Architecture Test1 of Test is
component DIV

port (CLK, RESET, TESTN: in std_logic;
ENA: out std_logic); end component;

type sw2_vector is array (Positive range <>, Positive range <>) of std_logic;

constant Input_vectors : sw2_vector :=

("100", "100", "100", "100", "110", "111", "011");

constant output_vectors : sw2_vector := ("0", "0", "0", "0", "0", "1");

constant strobe_delay : Time := 1 ns;

constant cycle_time : Time := 80 ns;

signal clock, RESET, TESTN, ENABLE : std_logic;

begin

 Apply : Process

 begin

 For J in INPUT_VECTORS' RANGE() loop

 Generate_pulse (

 Drive_signal => clock,

 signal_value => Input_vectors (J, 1),

 Drive_delay => 20 ns,

 Drive_width => 30 ns);

-- Procedure GENERATE_PULSE generates a pulse of specified

-- delay and width in package UTILS_PKG.

 RESET <= INPUT_VECTORS (J, 2);

 TESTN <= INPUT_VECTORS (J, 3);

 wait for cycle_time - strobe_delay;

 assert ENABLE = OUTPUT_VECTORS (J, 1)

 wait for strobe_delay;

 end loop;

end process Apply;

-- Entity under test.
D2: DIV port map (CLOCK, RESET, TESTN, ENABLE);
end Test1;

converting Real and Integer to time

variable INCR: integer; -- integer to time

...
EFF_A <= DRV_A after INCR * 1ns;

wait for $\left(\frac{\text{INCR}}{2}\right) * 1 \text{ ns};$

17ns / 1ns

-- Time to integer

52.44 * 1ns

-- Real to time.

Dumping Results into Text file

- Use WRITE procedure in predefined TEXTIO package.
- The following TEST BENCH saves data on every clock transition.

Library IEEE, BOOK_UB;

Use STD-TEXTIO.all;

USE std_logic_1164.all, BOOK_UB. UTILS-PKG.all;

-- package UTILS-PKG contains TO-STRING function.

Architecture TEST of TOP is

-- First define a file dump:

file DUMP: TEXT open WRITE-MODE is "|home\top.dump";

-- TEXT is a filetype predefined in TEXTIO package.

```

component Monitor port (InputA: mbit_vector (0 to 7);
InputB: in integer; InputC: in bit;
OutputA: out std_logic_vector (3 downto 0);
OutputB: out Real);
end component;
signal N1: bit_vector (0 to 7);
signal N2: integer;
signal ck: bit;
signal T1: std_logic_vector (3 downto 0);
signal T2: Real;

```

```

begin
  -- Design to be monitored.

```

```

D1: Monitor port map (InputA => N1, InputB => N2,
InputC => ck, OutputA => T1, OutputB => T2);

```

```

process

```

```

  variable BUF: line;
  -- line is an access type predefined in TEXTIO
  -- package.

```

```

  begin
    -- write header only in the beginning.
    WRITE (BUF, STRING ("Following is a dump of the
      circuit TOP: "));

```

```

    WRITELINE (DUMP, BUF);

```

```

  loop

```

```

    wait for ck;

```

```

WRITE (BUF, STRING'("TIME = "));
WRITE (BUF, NOW);
WRITE (BUF, STRING'("CLK = "));
WRITE (BUF, CLK);
WRITE (BUF, STRING'("N1 = "));
WRITE (BUF, N1);
WRITE (BUF, STRING'("N2 = "));
WRITE (BUF, N2);
WRITE (BUF, STRING'("T1 = "));
WRITE (BUF, TO_STRING (T1));

```

- write std-logic-vector value T1, function TO_STRING
- converts std-logic-vector to string.

```

WRITE (BUF, STRING'("T2 = "));
WRITE (BUF, T2);
Writeln (DUMP, BUF);

```

```

end loop;
end process;
end Test;

```

Output:

Following is a dump of circuit TOP:
 Time = 10ns, CLK = 0, N1 = 01101110, N2 = 7, T1 = 0ZZZ,
 T2 = 1.056000e + 02.
 ... and so on.

BIT, BOOLEAN, CHARACTER, STRING, BIT-VECTOR,
 INTEGER, REAL, TIME

- `WRITELINE` procedure also clears out the `BUF` variable so that next `WRITE` procedure will start writing into `BUF` from the beginning of a line.
- For different predefined `TYPE`, a different `WRITE` procedure is used.
- If real numbers are to be printed in decimal form a value for default parameter `DIGITS` in `WRITE` procedure has been specified.

`WRITE (L => BUF, VALUE => T2, DIGITS => 4);`

Converting Integer to string:

`IMAGE` attribute is used to convert integers to string. `WRITE` procedure defined in `TEXTIO` package can be used to convert integers to strings.

`WRITE (LINE-BUF, INT-VAL)` → (integer declared)

- Automatically allocates
- sufficient memory to store `INT-VAL`.

-- copy to string buffer:

`str_buf (1 to line_buf' length) := line_buf.all;`
 ↳ gives no. of characters that have been written to the line.

- If `line_buf` is not required anymore, then need
- to deallocate memory that it is pointing to:

`DEALLOCATE (LINE-BUF);`

Reading vectors from a Text File:

MS-16

- Read procedure available in TEXTIO package.
- write library and appropriate USE clause to stdlogic
- vector is available in package UTILS-PKG in
- library BOOK-LIB.

Architecture Test of Bottom is

- specify vector file.

file VECTORS: TEXT open READ-MODE is "alu3.vec";

- This file contains

- # This is a comment line

-- 0001 81 11100100

-- 0100 56 10000011

-- ...

- Declare the entity under test

component EUT port (EN1: in bit_vector (0 to 3);

EN2: in integer; EPX: in std_logic_vector (7 downto 0);

...);

end component;

constant vector-period: Time := 5 ns;

signal N1: bit_vector (0 to 3); signal N2: integer;

signal PARX: std_logic_vector (7 downto 0);

begin

Process

Variable BUF: LINE;

Variable N1-VAR: bit_vector (0 to 3);

Variable N2-VAR: integer;

Variable PARX-st: string (8 downto 1);

begin

while not ENDFILE (VECTORS) loop

Readline (VECTORS, BUF);

if BUF(1) = '#' then
next;

end if;

READ (BUF, N1-VAR);

READ (BUF, N2-VAR);

READ (BUF, PARX-str);

-- Apply values read to input signals of entity under test.

N1 <= N1-VAR;

N2 <= N2-VAR;

PARX <= To_stdlogicvector (PARX-str);

wait for vector-period;

end loop;

wait;

end process;

-- Instantiate the entity under test.

E1: EUT port map (EN1 => N1, EN2 => N2, EPX =>

PARX, ...);

...
end B Test;

In Read procedure signal cannot be passed in as a parameter to READ procedure.

Alternatively a procedure can be defined and then overloaded.

Procedure READ (L: inout LINE; value: out std_logic_vector)

∴ READ cannot predefined for std logic vector.

- For Reading from standard input
Keyword INPUT is used.

Readline (INPUT, BUF);

- BUF' LENGTH gives no. of characters still to be read, it is zero when end of line is reached.

while (BUF /= NULL) and (BUF' LENGTH /= 0) loop

Read (buf, INT-VAR);

...

end loop;

- Reads all integers till end of line is reached.

TEST BENCH FOR FULL ADDER CIRCUIT:

MS-19

Input Vectors are read from a file and results are dumped into a Text file.

```
Library BOOK-LIB;
Use BOOK-LIB. UTILS_PKG.all;    Use STD.TEXTIO.all;
entity FA-TEST is end;
Architecture TEST of FA-TEST is
    component Full-add
        port (CIN, A, B: in Bit; Cout, sum: out bit);
    end component;
    file Vec-file: Text
        open Read-mode is "/home/bhaskeer/MODELS/fadd.vec";
    file Result-file: Text
        open WRITE-mode is "\home\bhaskeer\MODELS\fadd.out";
    signal S: Bit_vector (0 to 2);
    signal Q: Bit_vector (0 to 1);
    begin
        FA: Full-add port map (S(0), S(1), S(2), Q(0), Q(1));
    process
        constant Prop-delay: Time := 2 ns;
        Variable Buf-in, buf-out: Line;
        variable out-str: bit_vector (0 to 1);
        variable num-vectors: integer := 0;
        variable Val-s: bit_vector (0 to 2);
```

begin

```
while not ENDFILE (VEC_FILE) loop
  READLINE (VEC_FILE, Buf-in);
```

```
-- Make sure Buf contains only 3 bits.
```

```
assert (Buf-in' length = 3)
```

```
  report "vector does not have 3 bits "
```

```
  severity ERROR;
```

```
  Read (Buf-in, Var-s);
```

```
  S <= Var-s;
```

```
wait for prop-delay;
```

```
Num-vectors := Num-vectors + 1;
```

```
write (Buf-out, num-vectors);
```

```
write (Buf-out, string'("."));
```

```
WRITE (Buf-out, S);
```

```
write (Buf-out, string'("---->"));
```

```
  out-sts := Q;
```

```
  write (Buf-out, out-sts);
```

```
whiteline (RESULT-file, Buf-out);
```

```
end loop;
```

```
report "Completed processing all vectors.";
```

```
-- stop simulation
```

```
wait;
```

```
end process;
```

```
end Test;
```

Input for Test Bench

000
001
010
011
100
101
110
111

Output produced by this test bench

1. 000 ----> 00
2. 001 ----> 01
3. 010 ----> 01
4. 011 ----> 10
5. 100 ----> 01
6. 101 ----> 10
7. 110 ----> 10
8. 111 ----> 11

Generate statement:

→ Used for conditional selection or replication during elaboration phase, for concurrent statements.

schemes:

- for generation scheme. (replication).
- if generation scheme. (conditional elaboration).

for generation syntax

generate-label : for generate-identifier in discrete/^{range} generate

[Block Declarations

begin]

concurrent-statements

end generate [generate-label];

- Values in discrete range is globally static, i.e. must be computable at elaboration time.
- Implicit declaration of Generate identifier
- Set of concurrent statements are replicated once for each value in discrete range.
- Type of identifier is defined by discrete range.

entity full_add4 is

```
port (A, B : in Bit_vector (3 downto 0); Cin : in Bit;
      sum : out Bit_vector (3 downto 0); Cout : out bit);
end full_add4;
```

Architecture Test of full_add4 is
component Full_adder

```
port (PA, PB, PC : in Bit; Pcout, Psum : out Bit);
end component;
```

```
signal CAR : Bit_vector (4 downto 0);
```

```
begin
```

```
  CAR(0) <= Cin;
```

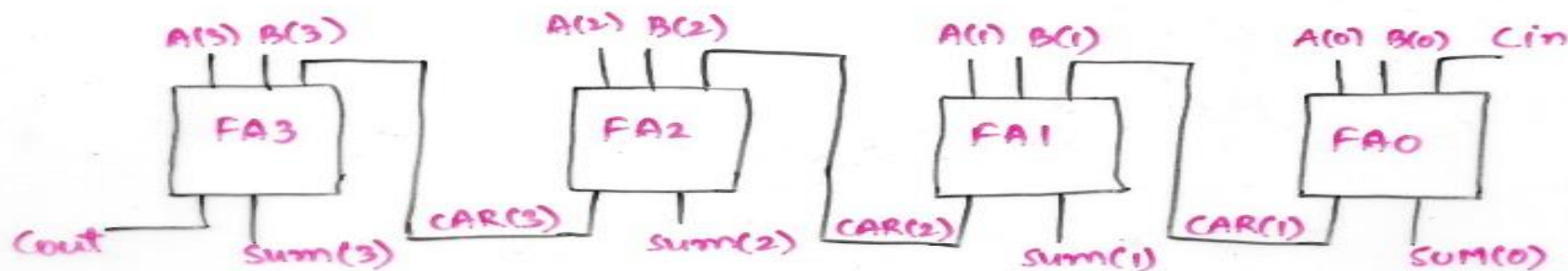
```
  GK: For K in 3 downto 0 generate
```

```
    FA: full_adder port map (CAR(K), A(K), B(K),
                             CAR(K+1), SUM(K));
```

```
  end generate GK;
```

```
  Cout <= CAR(4);
```

```
end test;
```



G1-3

During elaboration, generate statement is expanded to the following four Block statements.

G1K: Block

constant K: integer := 3;

begin

FA: full-adder port map (CAR(K), A(K), B(K), (CAR(K+1)
sum(K)));

end Block G1K;

G1K: Block

constant K: integer := 2;

begin

FA: full-adder port map (CAR(K), A(K), B(K), CAR(K+1),
sum(K));

end Block G1K;

⋮

similar entries for $K=1$ & $K=0$.

- Generate can have other concurrent assignment
i.e.

G12: For M in 3 downto 0 generate

Sum(M) <= A(M) XOR B(M) XOR CAR(M);

CAR(M+1) <= A(M) and B(M) and car(M);

end generate G12;

If generation scheme:syntax

```

generate-label : if expression generate
                [Block declaration
                 begin]
                concurrent-statements
                end generate [generate label];
  
```

- Expression must be globally static i.e. value must be computable at elaboration time.
- if-generate statement does not have an else or an elsif branch.

```

entity shifter4 is
  port (count, clock: in Bit; Q: buffer Bit-vector (0 to 3));
end shifter4;
  
```

Architecture test of shifter4 is

```

  component D_FF
    port (D, clk: in bit; Que: out bit);
  end component;
  
```

```

begin
  GK: for K in 0 to 3 generate
    GK0: if K = 0 generate
      D_FF1: D_FF port map (count, clock, Q(K));
    end generate
  end generate
end shifter4;
  
```

end generate GK0;

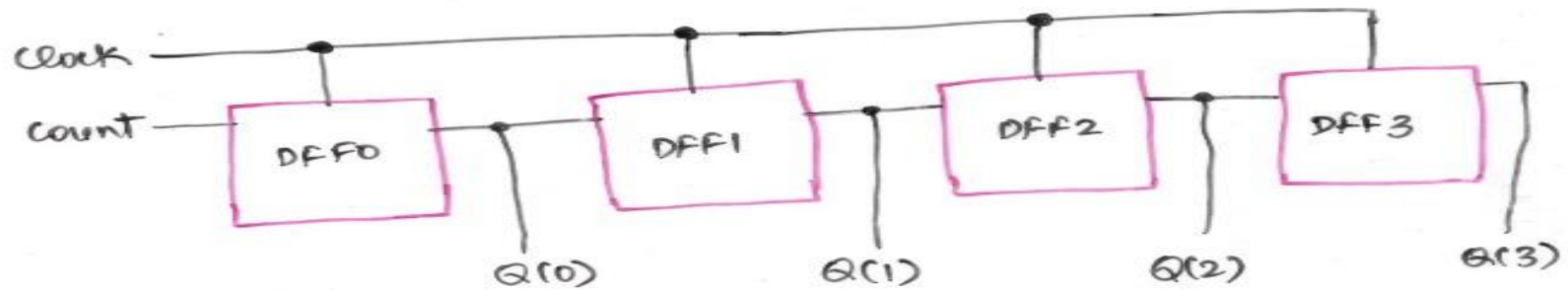
GK1-3: if $K > 0$ generate

D_FF2: D_FF port map ($Q(K-1)$, clock, $Q(K)$);

end generate GK1-3;

end generate GK;

end test;



4 Bit shifter

Package Declaration

- contains a set of declarations that may be shared by many design units.
- defines interface to the package.

package package-name is

package-item-declaration -->

- subprogram declaration.
- type, subtype declaration
- constant, signal, variable declaration.
- file, alias declaration.
- component declarations
- Attribute declarations & specifications.
- Use clause

end [package] [package-name];

Package example-pack is

constant LOW2HIGH : TIME := 20ns;

type MVL is (U, 0, 1, Z);

type MVL-VECTOR is array (Natural range <>) of MVL;

component NAND2

port (A, B: in MVL; C: out MVL);

end component;

end example-pack;

- Items declared in package declaration can be accessed by other design units using Library & Use clause.
- Above package does not contain any subprogram declaration or deferred constant, so package body is not necessary.

If set of declaration include

- functions
- subprogram
- deferred constant

Package Body is needed. , else not

Package Body:
contains -

- behaviour of subprograms, and
- the values of the deferred constants

Package body package-name is

Package Body item declarations ->

- subprogram bodies.
- subprogram declarations.
- type and subtype declarations.
- file and alias declarations
- use clause

end [package body] (package-name);

Note: Package name must be same as the name of corresponding package declaration.

Use work. synth-pack.all P-2
package program-pack is
constant prop_delay: Time;
function "and" (L, R: MVL) return
MVL;
procedure load (signal
Array_name: inout MVL-
vector);
start_bit, stop_bit, int_value: in integer);
end package program-pack;

Package body program- pack is

Use work. Tables. all;
function "and" (L, R: MVL) return MVL is
constant prop-delay : Time := 15 ns;

begin
return Table_AND (L, R);

-- Table_AND is constant defined in another package,
-- Tables.

end "and";

procedure LOAD (signal Array-name : inout MVL_vector;
start-bit, stop-bit, int-value : in integer) is

begin

-- Procedure behaviour here.

end LOAD;

end program-pack;

→ Items declared inside a package body has its scope restricted to be within the package body. This is similar to signals/variables declared in architecture body.

Difference between entity declaration/ package declaration

- Entity can have multiple architecture units with different names.
- Package declaration can have a single package body.

GENERIC & CONFIGURATIONS

GC-1

→ Used to pass certain types of information into a design description from its environment.

eg: entity AND-gate is

generic (N: Natural);
port (A: in bit_vector (1 to N); z: out bit);
end AND-gate;

Architecture gen of AND-gate is

begin

process (A)

variable AND_OUT : BIT;

begin

AND_OUT := '1';

for K in 1 to N loop

AND_OUT := AND_OUT and ACK);

exit when AND_OUT = '0';

end loop;

z <= AND_OUT;

end process;

end gen;

* size of input port has been modelled as generic.

→ A generic declares a constant object of mode GC-2 in.

→ The value of this constant can be specified in the one of the following:

→ Entity Declaration.

→ component declaration.

→ component instantiation.

- component specification.

- configuration declaration.

I Entity Declaration:

Entity NAND-gate is

generic (M: integer := 2);

```
port (A: in bit_vector (M downto 1);
```

```
z: out bit);
```

end NAND-gate;

→ Genetic must be determinable at elaboration time.

time.

→ Generics also use to model ranges of subtypes
e.g. subtype enum1 is integer range $\overline{\text{TOP}}$ down to 0
is a generic.

II COMPONENT DECLARATION

GC-3

Component AND_gate

generic (N: NATURAL := 5);

port [A: in Bit_vector (1 to N); Z: out Bit];

end component;

III component Instantiations

A: AND_gate generic map (N => 6)

port map (S1, S3);

Component label: component-name [generic map (generic-association-list)] [port map (port-association-list)]

CONFIGURATIONS

→ sometimes it may be convenient to specify multiple views for a single entity and use any one of these for simulation.

→ when it is desirable to associate a component with any one of a set of entities.

For the above reasons, configurations are required, but how they are declared & used in any program??

consider following component declaration.

```
component OR2:  
  port (A, B : in BIT;  Z : out BIT);  
end component;
```

Following may be the entity declaration.

I Entity OR-1A is
 port (N : out BIT; L, M : in BIT);
end OR-1A;

II Entity OR-1B is
 port (X, Y : in BIT; Z : out BIT);
end OR-1B;

A configuration is therefore used to bind the
following pairs:

- An architecture body to its entity declaration.
- A component with an entity.

Two ways to perform Binding

1. By using a configuration specification.
2. By using a configuration Declaration.

CONFIGURATION SPECIFICATION :

- used to bind components to specific entities stored in design Library.
- specification appears in the declarations part of the architecture.

eg: Library HS_LIB, CMOS_LIB;
 Entity FULL_ADDER is
 port (A, B, cin : in BIT;
 sum, cout : out BIT);
 end FULL_ADDER;

Architecture FA_STR of FULL_ADDER is
 component XOR2
 port (D1, D2 : in BIT; DZ : out BIT);
 end component;

component AND2
 port (z : out BIT; A0, A1 : in BIT);
 end component;

component OR2

port (N1, N2: in BIT; Z: out BIT);

end component;

-- for component specification

For X1, X2: XOR2

Use entity WORK. XOR2 (XOR2.BEH);

For A3: AND2

Use entity HS-LIB. AND2HS (AND2STR);

port map (HS-B => A1, HS-Z => Z, HS-A => A0);

Order & name are different.

For all: OR2

Use entity CMOS-LIB. OR2CMOS (OR2STR);

For others: AND2

Use entity WORK. A-GATE (A-GATE-BODY);

signal s1, s2, s3, s4, s5: BIT;

begin

X1: XOR2 port map (A, B, s1);

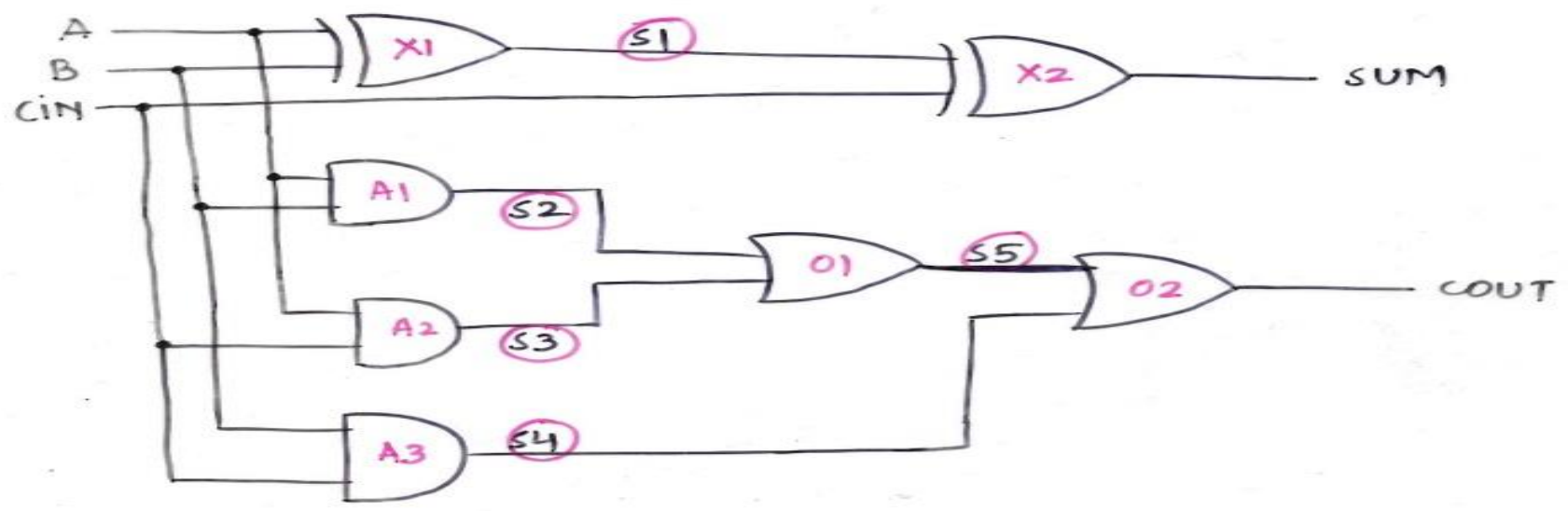
X2: XOR2 port map (s1, cin, sum);

A1: AND2 port map (s2, A, B);

A2: AND2 port map (s3, B, cin);

A3: AND2 port map (s4, A, cin);

```
O1: OR2 port map (S2, S3, S5);  
O2: OR2 port map (S4, S5, COUT);  
end FA_STR;
```



1-BIT FULL ADDER.

Syntax:

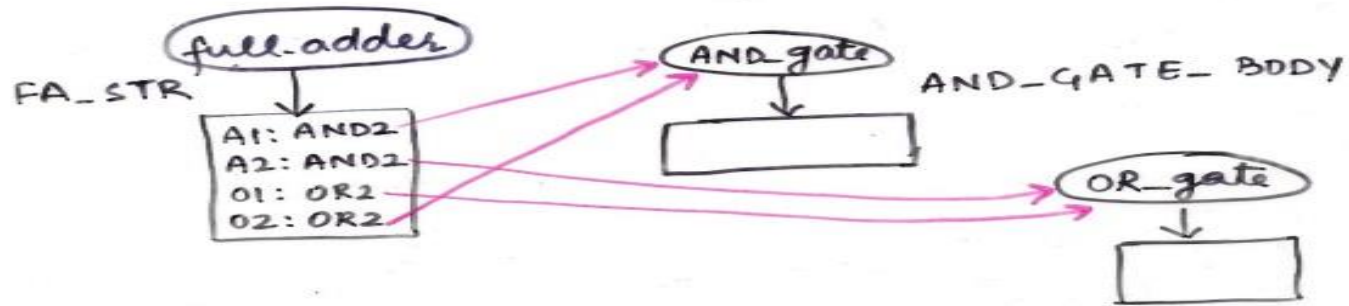
For list-of-comp.-Labels: component name
binding-indication;

syntax for Binding-indication

Use entity entity-name [(architecture-name)]
[generic map (generic-association-list)]
[port map (port-association-list)]

→ It is possible to bind different components to the same entity

eg: Instance of component AND2 binded with an entity called OR-gate. (A2)



→ A2 is made to behave as specified in the architecture of entity OR-gate.

→ Mainly used while debugging any program.

CONFIGURATION DECLARATION (Binding performed after architecture body has been written). GC-9

Syntax:

configuration ^{config-}name of entity-name is

Block-configuration;

end [configuration] [name];

Late Binding

Syntax for Block-configuration (Binding components in Block)

For Block-name

component-configurations

Block-configurations

end For;

can be:

- Architecture
- Block statement
- Generate stmt.

→ Name of arch. Body, Block stmt label or generate stmt label.

eg: Library CMOS-lib;

configuration fa-con of full-adder is

for FA_STR -- architecture name

For A1, A2, A3: AND2

Use entity CMOS-lib. B14 AND2 (AND2STR);

end for;

For ^{For others: OR2} all: XOR2

-- Use defaults.

Use entity DES-lib. STRXOR2 (XOR2-ARC);

end for;

end for;

end fa-con;

→ component configuration binds components that appears in a block to entities.

For Binding indication: (syntax)

GC-10-
to 10.4

1. For list-of-comp-labels: comp-name [binding-indication]
[Block-configuration]

end for;

2. Use configuration configuration-name

3. Use open

In 2, component instances are to be bound to a configuration of a lower-level entity as specified by configuration name.

In 3, the binding indication indicates that the binding is not yet specified and is to be deferred.

* The power of the configuration declaration lies in the fact that the subcomponents in the entire hierarchy of a design can be bound using a single configuration declaration.

entity FA is
port (A, B, Cin : in bit; Cout, Sum : out bit);
end FA;

Architecture FA1 of FA is
component HA

port (HA, HB : in bit; HS, HC : out bit);
end component;

component OR2

port (A, B : in bit; Z : out Bit);

end component;

signal S1, C1, C2 : Bit;

begin

HA1 : HA port map (A, B, S1, C1);

HA2 : HA port map (S1, Cin, Sum, C2);

O1 : OR2 port map (C1, C2, Cout);

end FA1;

entity Half-adder is
port (D0, D1 : in bit; S, C : out bit);
end Half-adder;

Architecture HA1 of Half-adder is

Component XOR2

port (X, Y: in bit; Z: out bit);
end component;

Component AND2

port (L, M: in bit; N: out bit);
end component;

begin

X1: XOR2 port map (D0, D1, S);

A1: AND2 port map (D0, D1, C);

end HA1;

-- Configuration declaration.

Library ECU;

Configuration full of FA is

For FA1

-- Architecture name.

for HA1, HA2: HA

Use entity work. Half-adder (HA1)

port map (D0 => HA, D1 => HB, S => HS,
C => HC);

For HA1

for all: XOR2

Use entity work. XOR2 (XOR2)

end for;

For A1: AND2

Use configuration ECL.AND2CON;

end for;

end for;

end for;

For O1: OR2

Use configuration WORK.OR2CON;

end for;

end for;

end full;

Default Rules

GC-11

1. An entity that is visible and has the same name as that of the component is used to bind to the instance; if no such entity exists, a default indication of "use open" is used.
2. The most recently analyzed architecture for that entity is used; it is an error if no architecture body exists.
3. For each port or generic in the component instance, there must exist a corresponding port or generic in the entity that matches by name, type and mode; any ports or generics in the entity that are unassociated are treated as open, if any matches fails, it is an error.

Conversion Functions:

→ Possible for port types of an entity to differ from the port types of the component being bound to.

Consider,

component COUNTER

port (CTR: inout bit_vector;
CLK, RST: in bit;
PAR: out bit);

end component;

Entity COUNTER is

port (A: inout std_logic_vector (3 downto 0);
CLOCK, RESET: in std_logic;
PARITY: out std_logic);

end COUNTER;

Conversion Function

For all: COUNTER use entity WORK. COUNTER
port map (TO_BITVECTOR(A) => TO_STDLOGICVECTOR(CTR),

CLOCK => TO_STDLOGIC(CLK),
RESET => TO_STDLOGIC(RST),

TO_BIT(PARITY) => PAR);

- conversion function specified on component port for an input port.
- for output port, it is specified with entity port.

DIRECT INSTANCE

GC-13

eg:

```
entity FULL_ADDER is
  port (A, B, cin : in BIT;
        cout, sum : out BIT);
end FULL_ADDER;
```

Architecture FA_WITH_HA of FULL_ADDER is
 signal s1, c1, c2 : BIT;

begin

```
HA1: Entity WORK_HA (HA_STR) port map (A, B, s1);
A3: entity HS_CIB_AND2HS (AND2STR)
  port map (s1, B, c1);
```

```

  ==
  ==
  ==
end FA_WITH_HA;
```

Incremental Binding:

→ Possible to delay the bindings of ports and generics and to override specified generics using incremental binding mechanism.

→ Conf. specification may not contain the complete binding information. Missing information:

- Unassociated ports and generics
- Open ports and generics.

eg: entity full_addr is
 port (A, B, Cin : in bit; Sum, Cout : out bit);
 end full_addr;

Architecture FA_struct of full_addr is
 component XOR2

port (O1, O2 : in bit; DZ : out bit);

end component;

component AND2 port (Z : out bit; A0, A1 : in bit);
 end component;

component OR2 port (N1, N2 : in bit; Z : out bit);
 end component;

For X1, X2 : XOR2

Use entity work. My_XOR2; -- No generic & port map
 -- No architecture name

For others: AND2 Use entity work. My_and2 (arch1)
 portmap (H5-B => A1, H5-A => open);

For all: OR2 Use entity work. My_or2

generic map (TPHL $\Rightarrow$ 2 ns, TPLH $\Rightarrow$ 3 ns);

GC-15

begin

$\equiv$

end FA_struct;

Entities are:

entity ^{my-}LXOR2 is

generic (TPHL, TPLH: Time);

port (XA, XB: in bit; XZ: out bit);

end my-XOR2;

entity My-AND2 is

port (HS-A, HS-B: in bit; HS-Z: out bit);

end My-AND2;

entity My-OR2 is

generic (TPHL, TPLH: Time);

port (N1, N2: in bit; Z: out bit);

end My-OR2;

Configuration FA of full-adder is

for FA_str

for X1, X2 : XOR2

port map (A0, A1, Z)

generic map (TPHL => 2 ns, TPHL => 5 ns);

end for;

for all : AND2

port map (HS_A => '1', HS_Z => Z);

end for;

end FA;

architecture ar-gen of en-gen is

component NAND-gate

generic (M: integer);

port (A: in bit_vector (M downto 1); z: out bit);

end component;

component AND-gate

generic (N: Natural := 5);

port (A: in bit_vector (1 to N); z: out bit);

end component;

signal S1, S2, S3, S4: Bit;

signal SB: bit_vector (2 downto 1);

signal SA: bit_vector (1 to 5);

signal SC: bit_vector (1 to 10);

signal SD: bit_vector (5 downto 0);

begin

N1: NAND-gate generic map (6) port map (SD, S1);

AN2: AND-gate generic map (N => 10) port map (SC, S3);

AN3: AND-gate port map (SA, S2);

N2: NAND-gate port map (SB, S2);

end ar-gen;

illegal →

Barrel Shifter

- Barrel shifter takes parallel data input and give shifted output either in left or right direction by a specific shift amount.
- When shift_by input is “000” it will place input data at the output without shifting.
- For specifying shifting direction shift_lt_rt pin is used.
- When it is ‘0’ the block will perform left shift operation and when it is ‘1’, it will perform right operation.

Barrel Shifter

Sr. No.	Name of the Pin	Direction	Width	Description
1	d_in	Input	8	data input
2	Shift_by	Input	3	shift amount
3	Rst_a	Input	1	Reset signal
4	Clk	Input	1	Clock signal
5	Shift_lt_rt	Input	1	'0'=shift left '1'=shift right
6	P_load	Input	1	'1'=receive input data '0'=discard input data
7	D_out	Output	8	Parallel data output

Barrel Shifter

Shift_by

shift_lt_rt = 0 for left shift and 1 for right shift

(2) (1) (0)

$\text{ctrl}(0) = \text{Shift_by}(0) \ \& \ \text{shift_lt_rt}$

0 0 0

for 00 and 01 means no shifting becoz Shift_by(0) = 0

0 0 1

0 1 0

ctrl(1) = (1) & shift_lt_rt for 00 and 01, Shift_by(1) = 0 means

0 1 1

Shift_by(0) has either 0 or 1 had already get the value x

1 0 0

1 0 1

1 1 0

1 1 1

Barrel shifter-CODE

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity barrel_shifter is
port (
    d_in      : in  std_logic_vector(7 downto 0); -- input vector
    d_out     : out std_logic_vector(7 downto 0); -- shifted output
    shift_lt_rt : in  std_logic;                  -- 0=>left_operation 1=>right_operation
    shift_by   : in  std_logic_vector(2 downto 0); -- shift amount
    clk       : in  std_logic;                    -- clock signal
    rst_a     : in  std_logic;                    -- reset signal
    p_load    : in  std_logic);                  -- parallel load
end barrel_shifter;
```

architecture beh of barrel_shifter is

begin -- beh

p1: process (clk,rst_a,shift_by,shift_lt_rt)

variable x,y : std_logic_vector(7 downto 0);

variable ctrl0,ctrl1,ctrl2 : std_logic_vector(1 downto 0);

begin -- process p1

ctrl0:=shift_by(0) & shift_lt_rt;

ctrl1:=shift_by(1) & shift_lt_rt;

ctrl2:=shift_by(2) & shift_lt_rt;

if(rst_a = '1') then

d_out<="00000000";

elsif(clk'event and clk = '1') then

if (p_load='0')then

assert(false) report "Parallel load low" severity warning;

elsif(shift_lt_rt='1')then

assert(false) report "right shift" severity warning;

elsif(shift_lt_rt='0')then

assert(false) report "left shift" severity warning;

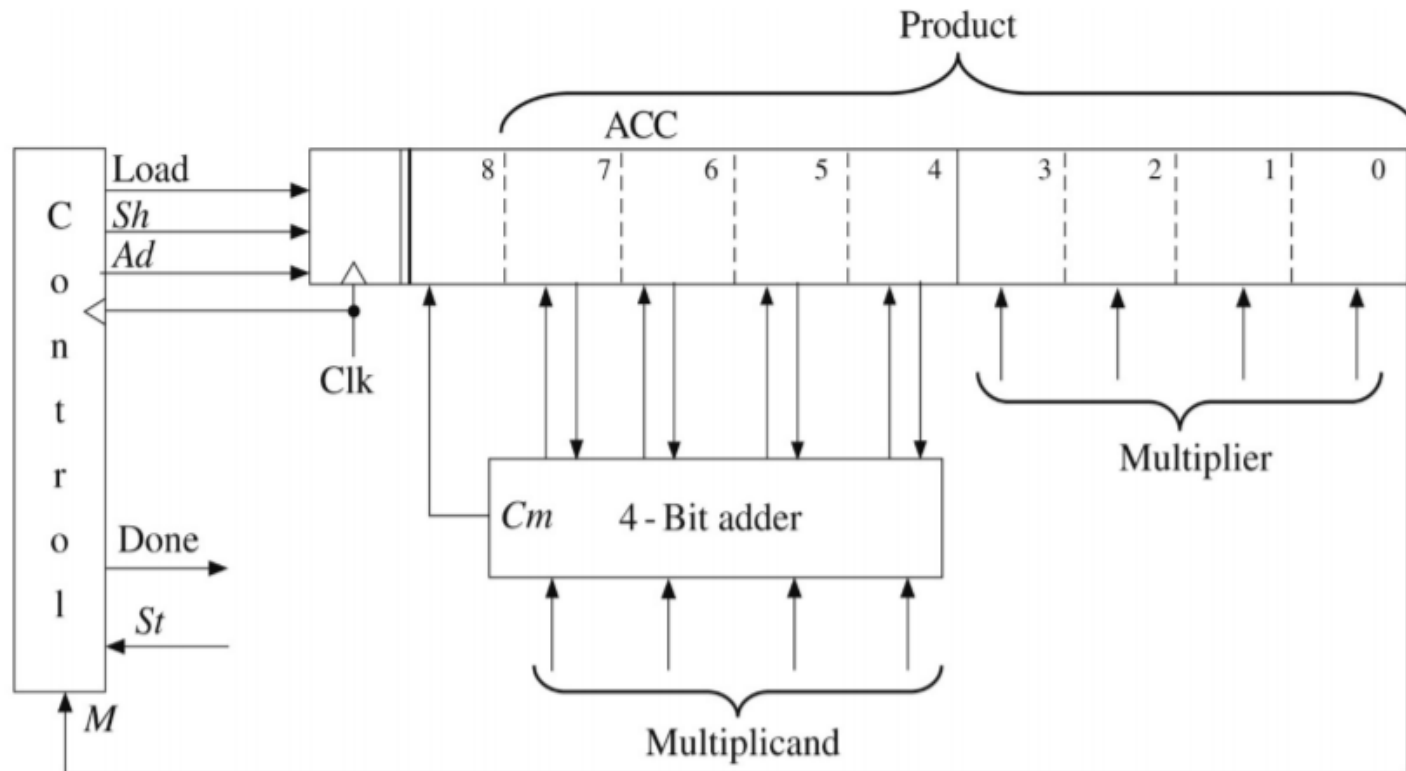
end if;

```

if p_load='1' then
  case ctrl0 is
    when "00"|"01" =>x:=d_in ;
    when "10" =>x:=d_in(6 downto 0) & d_in(7); --shift left by 1 bit
    when "11" =>x:=d_in(0) & d_in(7 downto 1); --shift right by 1 bit
    when others => null;
  end case;
  case ctrl1 is
    when "00"|"01" =>y:=x;
    when "10" =>y:=x(5 downto 0) & x(7 downto 6); --shift left by 2 bits
    when "11" =>y:=x(1 downto 0) & x(7 downto 2); --shift right by 2 bits
    when others => null;
  end case;
  case ctrl2 is
    when "00"|"01" =>d_out<=y ;
    when "10"|"11" =>d_out<= y(3 downto 0) & y(7 downto 4); --shift right/left by 4 bits
    when others => null;
  end case;
end if;
end if;
  end process p1;
end beh;

```

Binary Multiplier




```

entity mult4X4 is
    port (Clk, St: in bit;
          Mplier, Mcand : in bit_vector(3 downto 0);
          Done: out bit);
end mult4X4;

```

architecture behave1 of mult4X4 is

```

    signal State: integer range 0 to 9;
    signal ACC: bit_vector(8 downto 0);      --accumulator
    alias M: bit is ACC(0);                  --M is bit 0 of ACC

begin
    process
begin
    wait until Clk = '1';                    --executes on rising edge of clock
    case State is
        when 0=>                             --initial State
            if St='1' then
                ACC(8 downto 4) <= "00000";    --Begin cycle
                ACC(3 downto 0) <= Mplier;      --load the multiplier
            end if;
        end case;
    end process;
end;

```

```

    State <= 1;
end if;

when 1 | 3 | 5 | 7 =>          --"add/shift" State
    if M = '1' then           --Add multiplicand
        ACC(8 downto 4) <= add4(ACC(7 downto 4), Mcand, '0');
        State <= State + 1;
    else
        ACC <= '0' & ACC(8 downto 1); --Shift accumulator right
        State <= State + 2;
    end if;
when 2 | 4 | 6 | 8 =>          --"shift" State
    ACC <= '0' & ACC(8 downto 1); --Right shift
    State <= State + 1;
when 9 =>                      State <= 0;
end case;
end process;

```

```

Done <= '1' when State = 9 else '0'; --End of cycle
end behave1;

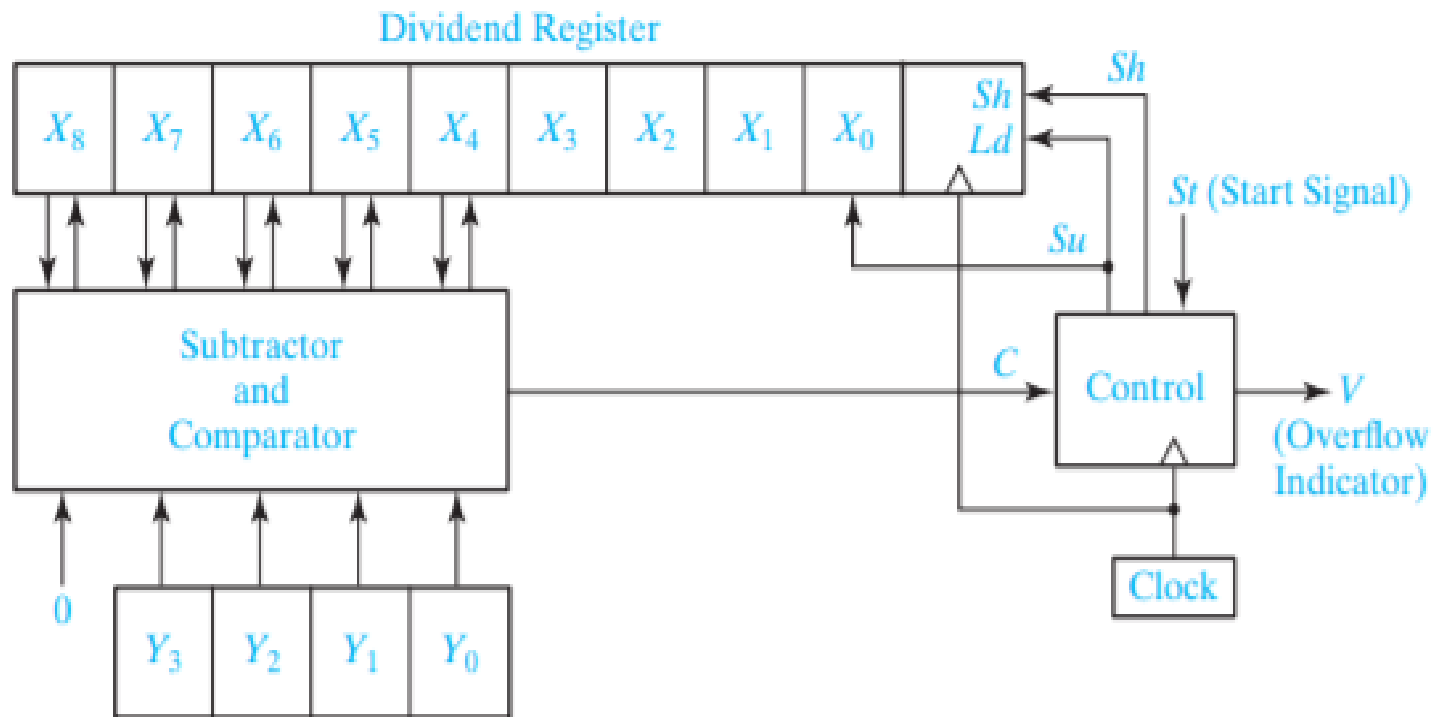
```

Division process

divisor	<u>1101</u>		1010	quotient
		<u>10000111</u>		dividend
		<u>1101</u>		
		0111		
		<u>0000</u>		
		1111		
		<u>1101</u>		
		0101		
		<u>0000</u>		
		0101		remainder

(135 ÷ 13 = 10 with
a remainder of 5)

Binary Divider




```

library ieee;
use ieee.std_logic_1164.all;

entity divider is
port (St, Clk: in std_logic;
dend: in std_logic_vector(7 downto 0);
dsor: in std_logic_vector(2 downto 0);
v: out std_logic;
qent: out std_logic_vector(4 downto 0));
end divider;
architecture beh of divider is
signal C, Sh, su, Ld: std_logic;
signal DendR: std_logic_vector(8 downto 0);
signal DsorR: std_logic_vector(2 downto 0);
signal Sub: std_logic_vector(4 downto 0);
signal State, nxState: integer range 0 to 6;
begin
Sub <= Add4 (DendR(8 downto 5), not('0' &
DsorR), '1');
C<=sub(4);
Qent<=DendR(4 downto 0);
Process (state, st, C)
Begin
V<= '0'; Sh<= '0'; Su<='0'; Ld<='0';
Case state is
When 0=> if (St='1') then Ld<='1'; nxState<='1';
Else nxstate<='0'; end if;

```

```

When 1=> if(C='1') then V<='1'; nxstate<='0';
Else Sh<='1'; nxState<=2; end if;
When 2|3|4|5 => if (C='1') then Su<='1';
nxstate<=State;
Else Sh<='1'; nxstate<=state + 1; end if;
When 6 => if (C='1') then Su<='1'; end if;
nxState<='0';
end case;
end process;
process (Clk)
begin
if (Clk='1' and Clk'event) then
state<=nxState;
if (Ld='1') then
DendR<='0' & dend;
DsorR<=dsor; end if;
If (Sh='1') then
DendR<= DendR (7 downto 0) & '0'; end if;
If (Su='1') then
DendR(8 downto 5) <=sub(3 downto 0);
DendR(0)<='1'; end if;
End if;
End process;
End divider;

```

Hamming code Encoder

```
ENTITY hamenc IS
```

```
    PORT(datain : IN BIT_VECTOR(0 TO 3); --d0 d1 d2 d3
```

```
          hamout : OUT BIT_VECTOR(0 TO 7)); --d0 d1 d2 d3 p0 p1 p2 p4
```

```
END hamenc;
```

```
ARCHITECTURE ver2 OF hamenc IS
```

```
    SIGNAL p0, p1, p2, p4 : BIT; --check bits
```

```
BEGIN
```

```
    --generate check bits
```

```
    p0 <= (datain(0) XOR datain(1)) XOR datain(2);
```

```
    p1 <= (datain(0) XOR datain(1)) XOR datain(3);
```

```
    p2 <= (datain(0) XOR datain(2)) XOR datain(3);
```

```
    p4 <= (datain(1) XOR datain(2)) XOR datain(3);
```

```
    --connect up outputs
```

```
    hamout(4 TO 7) <= (p0, p1, p2, p4);
```

```
    hamout(0 TO 3) <= datain(0 TO 3);
```

```
END ver2;
```

**DIGITAL DESIGN
BY
MORRIS MANO**

PAGE : 299

Hamming code Decoder- single error correction & double error detection

```
ENTITY hamdec IS
```

```
  PORT(hamin : IN BIT_VECTOR(0 TO 7); --d0 d1 d2 d3 p0 p1 p2 p4
```

```
  dataout : OUT BIT_VECTOR(0 TO 3); --d0 d1 d2 d3
```

```
  sec, ded, ne : OUT BIT); --diagnostic outputs
```

```
END hamdec;
```

```
ARCHITECTURE ver1 OF hamdec IS
```

```
  BEGIN
```

```
    PROCESS(hamin)
```

```
      VARIABLE syndrome : BIT_VECTOR(3 DOWNT0 0);
```

```
      BEGIN
```

```
        --generate syndrome bits
```

```
        syndrome(0) := (((((((hamin(0) XOR hamin(1)) XOR hamin(2)) XOR hamin(3))  
XOR hamin(4)) XOR hamin(5)) XOR hamin(6)) XOR hamin(7));
```

```
        syndrome(1) := (((hamin(0) XOR hamin(1)) XOR hamin(3)) XOR hamin(5));
```

```
        syndrome(2) := (((hamin(0) XOR hamin(2)) XOR hamin(3)) XOR hamin(6));
```

```
        syndrome(3) := (((hamin(1) XOR hamin(2)) XOR hamin(3)) XOR hamin(7));
```

Hamming code Decoder(cont.)

```
IF (syndrome = "0000") THEN --no errors
    ne <= '1';
    ded <= '0';
    sec <= '0';
    dataout(0 TO 3) <= hamin(0 TO 3);
ELSIF (syndrome(0) = '1') THEN --single bit error
    ne <= '0';
    ded <= '0';
    sec <= '1';
CASE syndrome(3 DOWNT0 1) IS
    WHEN "000"|"001"|"010"|"100" =>
dataout(0 TO 3) <= hamin(0 TO 3); -- parity errors
    WHEN "011" => dataout(0) <= NOT hamin(0);
dataout(1 TO 3) <= hamin(1 TO 3);
    WHEN "101" => dataout(1) <= NOT hamin(1);
dataout(0) <= hamin(0);
dataout(2 TO 3) <= hamin(2 TO 3);
    WHEN "110" => dataout(2) <= NOT hamin(2);
dataout(3) <= hamin(3);
dataout(0 TO 1) <= hamin(0 TO 1);
    WHEN "111" => dataout(3) <= NOT hamin(3);
dataout(0 TO 2) <= hamin(0 TO 2);
END CASE;
```


Hamming code Decoder(cont.)

--double error

```
ELSIF (syndrome(0) = '0') AND (syndrome(3 DOWNT0 1) /= "000") THEN  
  ne <= '0';  
  ded <= '1';  
  sec <= '0';  
  dataout(0 TO 3) <= "0000";  
END IF;  
END PROCESS;  
  
END ver1;
```

ALU

```
library IEEE; use IEEE.std_logic_1164.all; use IEEE.std_logic_unsigned.all;
```

```
entity ALU is
```

```
port ( a, b : in std_logic_vector (7 downto 0);
```

```
sel : in std_logic_vector(3 downto 0);
```

```
cin : in std_logic;
```

```
y : out std_logic_vector(7 downto 0));
```

```
end ALU;
```

```
architecture dataflow of ALU is
```

```
signal arith, logic : std_logic_vector (7 downto 0);
```

```
begin
```

```
with sel(2 downto 0) select
```

```
arith <= a when "000", a+1 when "001", a-1 when "010", b+1 when "100",
```

```
b-1 when "101", a+b when "110", a+b+cin when others;
```

ALU(CONT.)

with sel(2 downto 0) select

logic <= not a when "000", not b when "001", a and b when "010",
a or b when "011", a nand b when "100", a nor b when "101",
a xor b when "110", not (a xor b) when others;

with sel(1) select

y <= arith when '0',
logic when others;

end dataflow;

4 x 4 KEYBOARD ENCODER

begin

input : process (clk)

begin

if reset = '1' then

row_out <= "0001";

current_Input <= "00000";

previous_Input <= "00000";

elsif clk'event and clk = '1' then

previous_Input <= current_Input;

if columns = "1000" then

if row_out = "1000" then

current_Input <= "00001";

elsif row_out = "0100" then

current_Input <= "00010";

elsif row_out = "0010" then

current_Input <= "00011";

elsif row_out = "0001" then

current_Input <= "00100";

end if;

elsif columns = "0100" then

```
if row_out = "1000" then
    current_Input <= "00101";
elsif row_out = "0100" then
    current_Input <= "00110";
elsif row_out = "0010" then
    current_Input <= "00111";
elsif row_out = "0001" then
    current_Input <= "01000";
end if;
```

elsif columns = "0010" then

```
if row_out = "1000" then
    current_Input <= "01001";
elsif row_out = "0100" then
    current_Input <= "01010";
elsif row_out = "0010" then
    current_Input <= "01011";
elsif row_out = "0001" then
    current_Input <= "01100";
end if;
```

```
elsif columns = "0001" then
```

```
    if row_out = "1000" then
```

```
        current_input <= "01101";
```

```
    elsif row_out = "0100" then
```

```
        current_input <= "01110";
```

```
    elsif row_out = "0010" then
```

```
        current_input <= "01111";
```

```
    elsif row_out = "0001" then
```

```
        current_input <= "10000";
```

```
    end if;
```

```
else
```

```
    current_input <= "00000";
```

```
end if;
```

```

if row_out = "1000" then
    row_out <= "0100";
elsif row_out = "0100" then
    row_out <= "0010";
elsif row_out = "0010" then
    row_out <= "0001";
else
    row_out <= "1000";
end if;
if previous_Input = current_Input then
    LedTest <= "00000";
else
    LedTest <= current_Input;
end if;
previous_Input <= current_Input;

end if; end process;
rows <= row_out; end Behavioral;

```

Attributes (Refer 10.7 of Bhaskar)

An attribute is a value, function, type, range, signal, or a constant that can be associated with certain names within a VHDL description. These names could be among others, an entity name, an architecture name, a label, or a signal. For example, a record that contains the X and Y coordinates of a component placement could be associated with an entity name that describes the placement for that entity in a physical layout; a capacitance value can be associated with all signals of a specific type.

Predefined Attributes

There are five classes of predefined attributes.

1. Value attributes: these return a constant value
2. Function attributes: calls a function that returns a value
3. Signal attributes: creates a new signal
4. Type attributes: returns a type name
5. Range attributes: returns a range

Cont. (Take Examples from Book)

Value Attributes

If T is any scalar type or subtype,

- TLEFT: returns the left bound, that is, the leftmost value, of T.
- TRIGHT: returns the right bound, that is, the rightmost value, of T.
- THIGH: returns the upper bound, that is, the value at the highest position number, of T.
- TLOW: returns the lower bound, that is, the value at the lowest position number, of T.

Function Attributes

These attributes represent functions that are called to obtain a value. They are often used to convert values from an enumeration or physical type to an integer type. If T is a discrete type, a physical type, or a subtype,

- TPOS(V): returns the position number of the value V in the ordered list of values of T.
- T'VAUP): returns the value of the type that corresponds to position P.
- TSUCC(V): returns the value of the parameter whose position is one larger than the position of value V in T.
- TPRED(V): returns the value of the parameter whose position is one less than the position of value V in type T.
- TLEFTOF(V): returns the value of the parameter that is to the left of value V in type T.
- TRIGHTOF(V): returns the value of the parameter that is to the right of value V in type T.

Cont. (Take Examples from Book)

Signal Attributes

These attributes create new signals from the signals with which they are associated. These attributes, therefore, create implicit signals as compared to explicit signals that are created using signal declarations. If S is a signal object, then

- S'DELAYED (T): is a new signal that is the same type as signal S but delayed from S by time T. If T is not specified, a delay of 0ns is assumed.
- S'STABLE (T): is a boolean signal that is true when signal S has not had any event for time T. If T is not specified, it implies current delta.
- S'QUIET (T): creates a boolean signal that is true when S has not been active for time T.
- STRANSACTION: creates a bit type signal that toggles its value every time signal S becomes active.

Type Attributes

If T is any type or subtype, T'BASE, which is the only type attribute, returns the base type of T. This attribute cannot be used in expressions as such since it returns a base type but it can be used in conjunction with other attributes. For example, if

type ALU_OPS is (ADDOP, SUBOP, MULOP, DIVOP, ANDOP, NANDOP, OROP, NOROP); **subtype** ARITH_OPS is ALU_OPS **range** ADDOP to DIVOP;

then

ARITH_OPS'BASE is ALU_OPS

which is the base type, and it can be used in an expression such as

ARITH_OPS'BASE'LEFT which has the value ADDOP.

Cont. (Take Examples from Book)

Range Attributes

If A is a constrained array object, then

- A'RANGE (N): returns the Nth index range of A (N=1, if not specified).
- A'REVERSE.RANGE(N): returns the Nth index range reversed.

IEEE standard logic library

The following packages should be installed along with the VHDL compiler and simulator. The packages that you need, except for "standard", must be specifically accessed by each of your source files with statements such as:

```
library IEEE;  
use IEEE.std_logic_1164.all;  
use IEEE.std_logic_textio.all;  
use IEEE.std_logic_arith.all;  
use IEEE.numeric_bit.all;  
use IEEE.numeric_std.all;  
use IEEE.std_logic_signed.all;  
use IEEE.std_logic_unsigned.all;  
use IEEE.math_real.all;  
use IEEE.math_complex.all;
```

```
library STD;  
use STD.textio;
```

Cont.

STD_LOGIC_1164 package:

The IEEE library contains one of the most basic package, `std_logic_1164`. It contains the types `std_logic` (similar to `bit`) and `std_logic_vector` (similar to `bit_vector`) data types.

Digital signals are different in the sense that it can have values other than '0' or '1'. It can have undefined or high impedance conditions as shown in the flip flop structure in Hour 3.

The advantage of the `std_logic` types is that they can have values other than '0' and '1'. For example, an `std_logic` signal can also have undefined ('X') and high-impedance values ('Z'). The `std_logic_1164` package also redefines (“overloads”) the standard boolean operators (and, or, not, etc.) so that they work with `std_logic` signals.

USAGE: `IEEE.STD_LOGIC_1164.ALL`

Cont.

The package standard is predefined in the compiler.
Types defined include:

- bit bit_vector typical signals
- integer natural positive typical variables
- boolean string character typical variables
- real time delay_length typical variables

Cont.

package STANDARD is

-- predefined enumeration types:

- type BOOLEAN
- type BIT
- type CHARACTER
- type SEVERITY_LEVEL is (NOTE, WARNING, ERROR, FAILURE);

-- predefined numeric types:

-- Do INTEGER first to aid implicit declarations of "**".

- type INTEGER is range -2147483647 to 2147483647;
- type \$UNIVERSAL_INTEGER is range \$- to \$+;
- type \$UNIVERSAL_REAL is range \$-. to \$+.;
- function "*" (\$LEFT: \$UNIVERSAL_REAL; \$RIGHT: \$UNIVERSAL_INTEGER) return \$UNIVERSAL_REAL;
- function "*" (\$LEFT: \$UNIVERSAL_INTEGER; \$RIGHT: \$UNIVERSAL_REAL) return \$UNIVERSAL_REAL;
- function "/" (\$LEFT: \$UNIVERSAL_REAL; \$RIGHT: \$UNIVERSAL_INTEGER) return \$UNIVERSAL_REAL; type REAL is range \$-. to \$+.;

Cont.

--type FILE_OPEN_KIND is (READ_OPEN, WRITE_OPEN, APPEND_OPEN);

type FILE_OPEN_KIND is (READ_MODE, WRITE_MODE, APPEND_MODE);

type FILE_OPEN_STATUS is (OPEN_OK, STATUS_ERROR, NAME_ERROR, MODE_ERROR);

attribute FOREIGN: STRING;

-- predefined type

TIME: type TIME is range \$- to \$+ units fs; -- femtosecond

ps = 1000 fs; -- picosecond

ns = 1000 ps; -- nanosecond

us = 1000 ns; -- microsecond

ms = 1000 us; -- millisecond

sec = 1000 ms; -- second

min = 60 sec; -- minute hr = 60 min;

-- hour; end units;

Cont.

```
-- subtype used internally for checking time expressions for non-negativeness:
•  subtype $NATURAL_TIME is TIME range 0 sec to TIME'HIGH;
•  subtype DELAY_LENGTH is TIME range 0 fs to TIME'HIGH;
-- function that returns the current simulation time: impure function NOW
return TIME;
-- predefined numeric
•  subtypes: subtype NATURAL is INTEGER range 0 to INTEGER'HIGH;

subtype POSITIVE is INTEGER range 1 to INTEGER'HIGH;

-- predefined array types: type STRING is array (POSITIVE range <>) of
CHARACTER;
type BIT_VECTOR is array (NATURAL range <>) of BIT;
```

Cont.

```
-- The rest of this package is SVEN specific stuff required to
make this implementation go.
-- Note that all things are declared use leading $ characters, so we
don't trample the user's name space.
-- attribute $BUILTIN: BOOLEAN;
procedure $RTINDEX (I: NATURAL; FIRSTARG: INTEGER;
PASSAP,SAVEFREGS: BOOLEAN);
procedure $RTSYMBOL (S: STRING; FIRSTARG: INTEGER;
PASSAP,SAVEFREGS: BOOLEAN);
attribute $BUILTIN of all: function is TRUE;
attribute $BUILTIN of all:
procedure is TRUE;
end STANDARD;
```